

Supplementary Material for ProtGram-DirectGCN

1 APPENDIX A: ALGORITHMS

This section provides the pseudocode for the main components of our proposed pipeline, including the hierarchical graph construction and the forward pass of the *ProtGramDirectGCN* model.

Algorithm 1 Hierarchical N-gram Graph Construction

▷ Constructs a hierarchy of n-gram graphs $G_1, \dots, G_{N_{max}}$ from a protein sequence corpus.

Require: Corpus of protein sequences S_{corpus} , Maximum n-gram size N_{max}

Ensure: A set of n-gram graphs $\{G_1, \dots, G_{N_{max}}\}$

```

1: for  $n = 1 \dots N_{max}$  do
2:   Initialize unique n-gram set  $V_n \leftarrow \emptyset$ 
3:   Initialize edge counts  $W\_counts\_n \leftarrow$  empty map

   // Phase 1: Identify all unique n-grams of length n
4:   for each protein sequence  $R$  in  $S_{corpus}$  do
5:     for  $i = 0 \dots \text{length}(R) - n$  do
6:        $ngram \leftarrow R[i : i + n]$ 
7:       Add  $ngram$  to  $V_n$ 
8:     end for
9:   end for
10:  Create integer mapping  $node\_to\_idx\_n$  for all n-grams in  $V_n$ .

   // Phase 2: Count transitions between n-grams
11:  for each protein sequence  $R$  in  $S_{corpus}$  do
12:    for  $i = 0 \dots \text{length}(R) - n - 1$  do
13:       $u \leftarrow R[i : i + n]$ 
14:       $v \leftarrow R[i + 1 : i + 1 + n]$ 
15:      if  $u \in V_n$  and  $v \in V_n$  then
16:        Increment count for edge  $(u, v)$  in  $W\_counts\_n$ 
17:      end if
18:    end for
19:  end for

   // Phase 3: Assemble the graph object  $G_n$ 
20:  Create edge list  $E_n$  from  $W\_counts\_n$  using integer IDs from  $node\_to\_idx\_n$ .
21:   $G_n \leftarrow \text{InstantiateGraph}(V_n, E_n)$ 
22:  Store  $G_n$ 
23: end for
24: return  $\{G_1, \dots, G_{N_{max}}\}$ 

```

Algorithm 2 *DirectGCNLayer* - Node-Level Feature Propagation (Per-Dimension View)

▷ Computes output features for all nodes, showing the per-dimension logic.

Require: Node features $H^{(l)}$ ($N \times F_{in}$), Prop. matrices $\mathcal{A}_{in}, \mathcal{A}_{out}, \tilde{\mathcal{A}}_{undir}$ **Require:** Layer parameters: $W_{main,*}, b_{main,*}, W_{shared}, b_{shared,*}, C_*, b_{const}$ **Ensure:** Pre-activation output features $H^{(l+1)'} (N \times F_{out})$

```

1: Initialize  $H^{(l+1)'}$  as a zero matrix of shape  $(N \times F_{out})$ 
2: for each node  $u = 0 \dots N - 1$  do
3:   for each output dimension  $d = 0 \dots F_{out} - 1$  do

      // Initialize accumulators for node  $u$ , dimension  $d$ 
4:     agg_main_in  $\leftarrow 0$ , agg_main_out  $\leftarrow 0$ , agg_main_undir  $\leftarrow 0$ 
5:     h_shared  $\leftarrow 0$ 

      // 1. Aggregate messages from neighbors
6:     for each input dimension  $f = 0 \dots F_{in} - 1$  do
7:       for each neighbor node  $v = 0 \dots N - 1$  do
8:         agg_main_in  $\leftarrow$  agg_main_in  $+$   $\mathcal{A}_{in,uv} \cdot H_{vf}^{(l)} \cdot W_{main,in,fd}$ 
9:         agg_main_out  $\leftarrow$  agg_main_out  $+$   $\mathcal{A}_{out,uv} \cdot H_{vf}^{(l)} \cdot W_{main,out,fd}$ 
10:        agg_main_undir  $\leftarrow$  agg_main_undir  $+$   $\tilde{\mathcal{A}}_{undir,uv} \cdot H_{vf}^{(l)} \cdot W_{main,undir,fd}$ 
11:      end for
12:    end for

      // 2. Compute shared MLP transformation
13:    for each input dimension  $f = 0 \dots F_{in} - 1$  do
14:      h_shared  $\leftarrow$  h_shared  $+$   $H_{uf}^{(l)} \cdot W_{shared,fd}$ 
15:    end for

      // 3. Combine components for each channel
16:    H_in_d  $\leftarrow$  (agg_main_in  $+$   $b_{main,in,d}$ )  $+$  (h_shared  $+$   $b_{shared,in,d}$ )
17:    H_out_d  $\leftarrow$  (agg_main_out  $+$   $b_{main,out,d}$ )  $+$  (h_shared  $+$   $b_{shared,out,d}$ )
18:    H_undir_d  $\leftarrow$  (agg_main_undir  $+$   $b_{main,undir,d}$ )  $+$  (h_shared  $+$   $b_{shared,undir,d}$ )

      // 4. Apply gating and final combination
19:    directed_signal  $\leftarrow C_{directed,u} \cdot (C_{in,u} \cdot H\_in\_d + C_{out,u} \cdot H\_out\_d)$ 
20:    undirected_signal  $\leftarrow C_{undirected,u} \cdot H\_undir\_d$ 
21:     $H_{ud}^{(l+1)'} \leftarrow C_{all,u} \cdot (\text{directed\_signal} + \text{undirected\_signal}) + b_{const,ud}$ 
22:  end for
23: end for
24: return  $H^{(l+1)'}$ 

```

2 A REVIEW OF CONVOLUTIONAL METHODS FOR DIRECTED GRAPHS

Extending Graph Convolutional Networks (GCNs) to directed graphs is a significant challenge, as the standard spectral GCN formulation relies on the symmetric Laplacian of an undirected graph. Research to address this has broadly followed three themes: adapting spectral theory, redesigning spatial convolutions, and leveraging higher-order structures.

Spectral approaches aim to redefine the graph Laplacian to handle directedness. An early attempt by Ma et al. (2019) constructed a symmetric Laplacian using the graph's transition matrix and its stationary distribution (the Perron vector), but this is computationally expensive. A more recent innovation is MagNet Zhang et al. (2021), which employs a complex Hermitian matrix called the magnetic Laplacian, $L_N^{(q)} := I - (D_s^{-1/2} A_s D_s^{-1/2}) \odot \exp(i\Theta^{(q)})$, where directionality is elegantly encoded in the phase of complex entries, guaranteeing real eigenvalues suitable for spectral filtering.

A second, more intuitive theme involves spatial approaches that redefine the neighborhood aggregation process. The most common strategy, seen in models like Tong et al. (2020)'s, is to explicitly separate the aggregation of features from in-neighbors and out-neighbors. A generalized form of this operation is $H' = \sigma(\tilde{A}_{in} H W_{in} + \tilde{A}_{out} H W_{out})$, where $\tilde{A}_{in}$ and $\tilde{A}_{out}$ are normalized adjacency matrices for incoming and outgoing edges. This method is straightforward to implement and scale but has a localized receptive field.

Finally, some research has moved beyond simple edges to consider higher-order connectivity patterns. MotifNet Monti et al. (2018), uses small, recurring directed subgraphs called motifs to define multiple anisotropic views of the graph, allowing the model to capture more complex structural information. Our *ProtGramDirectGCN* builds upon the principles of spatial methods by explicitly handling incoming, outgoing, and undirected information channels. However, it introduces a unique hybrid GCN-MLP layer architecture with a gating mechanism, allowing the model to learn the relative importance of each information channel and a shared feature transformation for a more expressive aggregation scheme.

3 DATA PREPARATION

SQL statments

4 COMPUTATIONAL CAPACITY

REFERENCES

- [Dataset] Ma, Y., Hao, J., Yang, Y., et al. (2019). Spectral-based Graph Convolutional Network for Directed Graphs. doi:10.48550/arXiv.1907.08990. ArXiv:1907.08990
- [Dataset] Monti, F., Otness, K., and Bronstein, M. M. (2018). MotifNet: a motif-based Graph Convolutional Network for directed graphs. doi:10.48550/arXiv.1802.01572. ArXiv:1802.01572
- [Dataset] Tong, Z., Liang, Y., Sun, C., et al. (2020). Directed Graph Convolutional Network
- [Dataset] Zhang, X., He, Y., Brugnone, N., et al. (2021). MagNet: A Neural Network for Directed Graphs